Currying, Partial Application & Composition

Currying/Composition · Build price pipeline from small functions · Build with Waqa

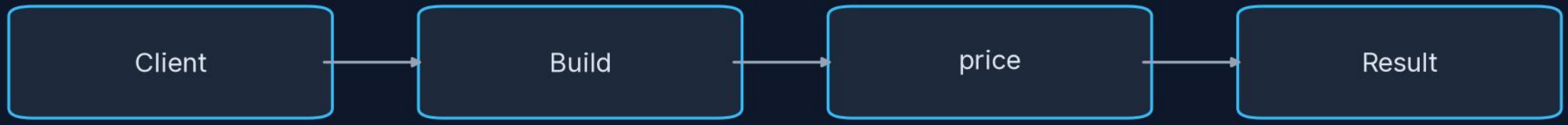
Today's Hook

- Problem: Build price pipeline from small functions
- Pattern: Currying/Composition — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Build price pipeline from small functions
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Currying/Composition: Build price pipeline from small functions

Real-World Scenario

- Running example: Build price pipeline from small functions
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.functions;

import com.buildwithwaqar.patterns.functional.functions.Fn;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: build a pricing pipeline and memoize hot square lookups. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Currying/Memoization problem ===");
    var add = Fn.curry((Integer x, Integer y) -> x + y);
    System.out.println("curry=" + add.apply(2).apply(40));
    var hits = new AtomicInteger();
    var memo = Fn.memoize((Integer x) -> { hits.incrementAndGet(); return x * x; });
    System.out.println("memo=" + memo.apply(12) + "," + memo.apply(12) + " hits=" + hits.get());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.functions;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.function.Function;

public final class Fn {
    private Fn() {}

    public static <A, B, C> Function<A, Function<B, C>> curry(java.util.function.BiFunction<A, B, C> f) {
        return a -> b -> f.apply(a, b);
    }

    public static <A, B, C> Function<A, C> compose(Function<B, C> g, Function<A, B> f) {
        return a -> g.apply(f.apply(a));
    }

    public static <A, B> Function<A, B> memoize(Function<A, B> f) {
        Map<A, B> cache = new ConcurrentHashMap<>();
        return a -> cache.computeIfAbsent(a, f);
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.functions;

import com.buildwithwaqar.patterns.functional.functions.Fn;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: build a pricing pipeline and memoize hot square lookups. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Currying/Memoization problem ===");
    var add = Fn.curry((Integer x, Integer y) -> x + y);
    System.out.println("curry=" + add.apply(2).apply(40));
    var hits = new AtomicInteger();
    var memo = Fn.memoize((Integer x) -> { hits.incrementAndGet(); return x * x; });
    System.out.println("memo=" + memo.apply(12) + "," + memo.apply(12) + " hits=" + hits.get());
  }
}
```

Walkthrough Output

```
=== Currying/Memoization problem ===
curry=42
memo=144,144 hits=1
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Currying/Composition.

Related Patterns

- Contrast Currying/Composition with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Currying/Composition → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.